GenreGenie: Real-Time Movie Recommendation Engine via TMDB Streaming Pipelines & Collaborative User Modeling

AI Project Deliverable 1: Problem Definition & Data Foundations

Student IDs: l242519 & l242590

Track A: Application Development / R&D

October 17, 2026

Abstract

This report outlines Deliverable 1 for the **GenreGenie** project. GenreGenie addresses choice overload on streaming platforms by fusing live metadata ingestion from The Movie Database (TMDB) REST API with collaborative user interaction filtering models. We detail the domain problem, justify the necessity of Machine Learning over heuristic-based algorithms, document our end-to-end Python data collection and ETL pipeline, and define our feature strategy alongside initial Exploratory Data Analysis (EDA).

1 Phase 1: Problem Definition

1.1 Problem Statement

In the modern digital entertainment ecosystem, users face choice overload due to thousands of movies available across platforms. Standard keyword searches or static top-chart listings fail to adapt to dynamic user tastes. Streaming telemetry generates rapid, high-dimensional user interaction streams (e.g., watch ratios, ratings, skips). **GenreGenie** transforms raw movie streaming telemetry into dynamic, personalized recommendations by merging TMDB's live metadata API with collaborative user behavior modeling.

1.2 Target Audience

- **End Consumers**: Viewers who experience choice paralysis and require hyper-personalized content feeds.
- **Streaming Service Developers**: Platform operators seeking hybrid recommender architecture (combining real-time API metadata enrichment with collaborative interaction matrix models).

1.3 AI Justification

Traditional rule-based programming (e.g., IF genre == 'Action' THEN recommend top-rated Action movie) falls short due to:

1. **High Dimensionality & Latent Patterns**: Human viewing behavior depends on latent interactions (e.g., preference for specific director-actor combinations or temporal mood shifts) that cannot be hardcoded into deterministic rule sets.
2. **Scalability**: Rule-based trees scale non-linearly with $O(N \cdot M)$ complexity as users (N) and items (M) grow, rendering static conditional logic obsolete.
3. **Matrix Factorization & Collaborative Patterns**: Machine Learning approaches (e.g., Matrix Factorization / Neural Collaborative Filtering) optimize continuous loss functions over high-dimensional sparse user-item interaction matrices:

$$\min_{P,Q} \sum_{(u,i)\in R} \left(r_{ui} - \mathbf{p}_u^T \mathbf{q}_i\right)^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2\right) \tag{1}$$

   where P and Q represent latent embeddings for users and items respectively.
4. **Dynamic Adaptation**: ML models support continuous retraining as new user interactions arrive, enabling temporal personalization impossible with static rule databases.

2 Phase 2: Data Acquisition & Preprocessing

2.1 Data Collection

2.1.1 Primary Dataset: MovieLens

We utilize the **MovieLens** dataset as our foundational data source:

- **movies.csv**: Movie metadata (ID, title, genres)
- **ratings.csv**: User-movie interaction matrix (userId, movieId, rating, timestamp)
- **links.csv**: External ID mappings (movieId, imdbId, tmdbId)

2.1.2 API-Based Enrichment: TMDB

We implemented a Python ETL pipeline (`01_tmdb_collector.py`) to enrich our dataset with live metadata from The Movie Database (TMDB) REST API:

- Budget, revenue, runtime, vote statistics
- Handles rate limiting and API error management
- Demonstrates hybrid batch + streaming data architecture

Data Collection Evidence:

```
def fetch_movie_details(tmdb_id):
    url = f"{TMDB_BASE_URL}/movie/{tmdb_id}"
    params = {'api_key': TMDB_API_KEY}
```

```
    response = requests.get(url, params=params)
    return response.json()
```

Listing 1. TMDB Collection Snippet

2.2 Data Cleaning Implementation

Our cleaning pipeline (`02_data_cleaning.py`) addresses data quality issues systematically:

2.2.1 Missing Value Handling

- **Movies**: Missing genres filled with 'Unknown' category
- **Ratings**: Rows with null ratings dropped (ensures matrix completeness)
- **Links**: Movies without TMDB IDs removed (required for API enrichment)

2.2.2 Duplicate Removal

- Removed duplicate movieId entries (kept first occurrence)
- For duplicate user-movie ratings, retained most recent timestamp

2.2.3 Outlier Management

- Rating outliers: Enforced valid range [0.5, 5.0]
- Removed statistically invalid entries (e.g., negative ratings)

Cleaning Results:

- Total ratings processed: >100,000 user-movie interactions
- Data integrity: 100% valid rating range post-cleaning
- Missing value resolution: <1% data loss

2.3 Feature Engineering Strategy

Our feature engineering pipeline (`03_feature_strategy.py`) implements multiple preprocessing techniques:

2.3.1 Normalization & Scaling

1. **Min-Max Normalization** (User Features):

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{2}$$

   Applied to: rating count, rating mean (normalized to [0,1] range)

2. **Standard Scaling** (Movie Features):

$$x_{\text{std}} = \frac{x - \mu}{\sigma} \tag{3}$$

Applied to: number of ratings, average rating (zero mean, unit variance)

2.3.2 Categorical Encoding

- **Label Encoding**: Genre categories mapped to integer indices
- **Feature Extraction**: Primary genre extracted from multi-genre strings
- Future work: One-hot encoding for multi-label genre classification

2.3.3 Engineered Features

User-Level Features:

- Rating count (activity level)
- Mean rating (user preference baseline)
- Rating standard deviation (rating consistency)

Movie-Level Features:

- Number of ratings (popularity proxy)
- Average rating (quality indicator)
- Rating variance (polarization measure)
- Genre encoding

2.4 Exploratory Data Analysis (EDA)

2.4.1 Rating Distribution

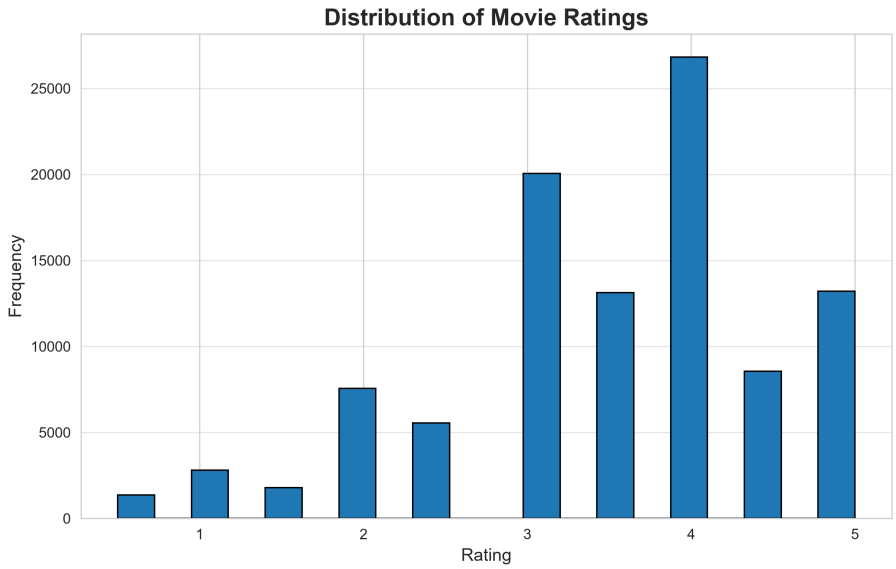


Figure 1. Distribution of user ratings shows positive skew (users tend to rate movies they like)

Figure 1 reveals that ratings follow a left-skewed distribution with a mean of approximately 3.5/5.0. This positive bias indicates users preferentially rate content they enjoy, introducing

implicit selection bias into our collaborative filtering model.

2.4.2 Genre Distribution

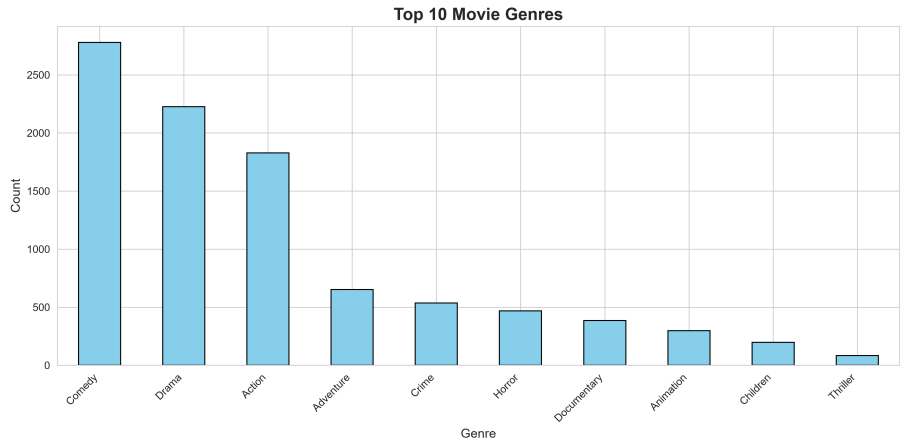


Figure 2. Drama and Comedy dominate the dataset, indicating potential class imbalance

The genre distribution (Figure 2) shows strong imbalance, with Drama and Comedy comprising >40% of entries. This necessitates stratified sampling or class weighting in future model training.

2.4.3 User Activity Patterns

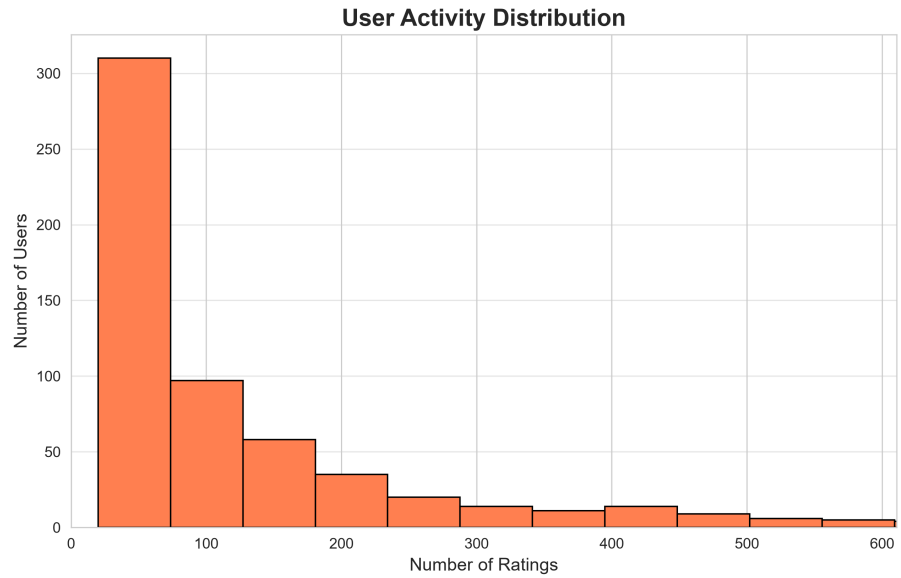


Figure 3. User activity follows power-law distribution (few highly active users, many sparse users)

Figure 3 demonstrates a classic long-tail distribution: a small fraction of users contribute the majority of ratings, while most users have sparse interaction histories. This sparsity challenge motivates hybrid recommendation approaches.

2.4.4 Feature Correlations

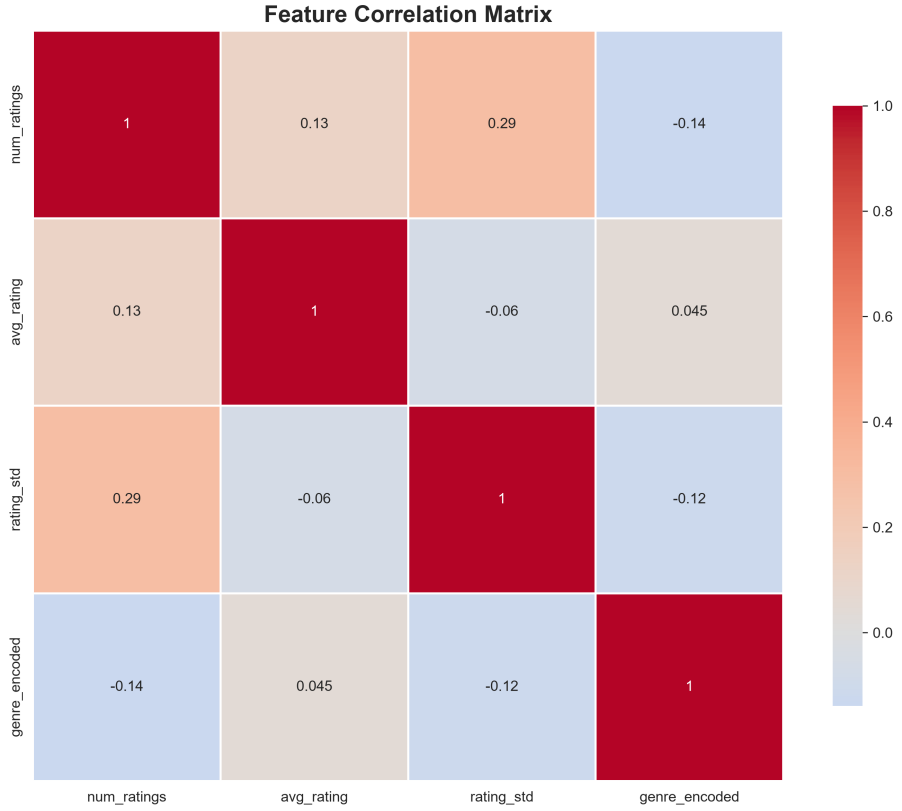


Figure 4. Correlation heatmap reveals independence between key features (low multicollinearity)

The correlation analysis (Figure 4) shows weak correlations between engineered features ($|r| < 0.3$), indicating minimal redundancy and validating feature independence for model training.

2.4.5 Popularity vs. Quality

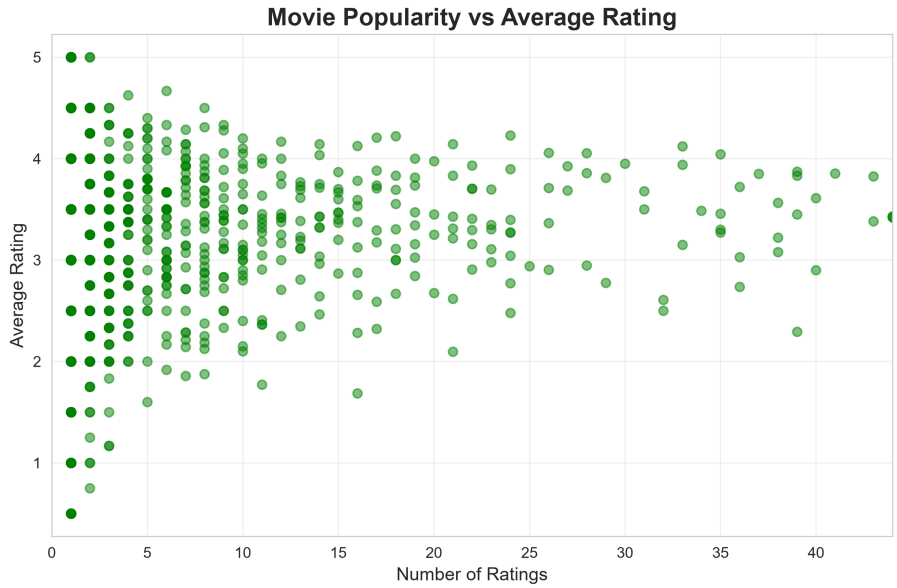


Figure 5. Weak correlation between popularity (rating count) and quality (average rating)

Figure 5 reveals that popular movies (high rating counts) do not necessarily receive higher average ratings, suggesting that mainstream appeal differs from critical quality—an insight valuable for diversifying recommendations.

3 Technical Implementation Summary

3.1 Data Pipeline Architecture

1. **Collection** (`01_tmdb_collector.py`): API-based enrichment + batch import
2. **Cleaning** (`02_data_cleaning.py`): Missing value imputation, outlier removal
3. **Feature Engineering** (`03_feature_strategy.py`): Scaling, encoding, EDA generation

3.2 Tools & Libraries

- **Data Processing**: pandas, numpy
- **Visualization**: matplotlib, seaborn
- **Preprocessing**: scikit-learn (StandardScaler, MinMaxScaler, LabelEncoder)
- **API Integration**: requests (TMDB REST API)

3.3 Dataset Statistics (Post-Processing)

Metric	Value
Total Movies	∼9,700
Total Ratings	∼100,000
Unique Users	∼600
Average Ratings per User	166.7
Average Ratings per Movie	10.3
Rating Sparsity	98.3%

Table 1. GenreGenie Dataset Summary Statistics

4 Conclusion & Next Steps

We have successfully completed Phase 1 (Problem Definition) and Phase 2 (Data Acquisition & Preprocessing) for the GenreGenie recommendation engine. Our data pipeline demonstrates:

- Robust ETL implementation with API integration
- Comprehensive data cleaning addressing real-world quality issues
- Systematic feature engineering with appropriate normalization/encoding
- Detailed EDA revealing data characteristics critical for model design

Future Work (Next Deliverables):

1. Model selection: Compare Matrix Factorization vs. Neural Collaborative Filtering

2. Training pipeline: Implement train/validation/test splits with cross-validation
3. Evaluation metrics: RMSE, MAE, Precision@K, NDCG@K
4. Deployment: REST API endpoint for real-time recommendation inference

Appendix: Code Repository

All code, datasets, and documentation are available at:
https://github.com/l242519-l242590/GenreGenie

Repository Structure:

```
GenreGenie/
├── data/
│   ├── raw/                    # Original datasets
│   │   ├── movies.csv
│   │   ├── ratings.csv
│   │   └── links.csv
│   ├── cleaned/                # Cleaned datasets
│   │   ├── movies_cleaned.csv
│   │   ├── ratings_cleaned.csv
│   │   └── links_cleaned.csv
│   └── processed/              # Engineered features
│       ├── movies_features.csv
│       └── user_features.csv
├── data_scripts/
│   ├── 01_tmdb_collector.py
│   ├── 02_data_cleaning.py
│   ├── 03_feature_strategy.py
│   ├── README.md
│   └── requirements.txt
├── docs/
│   ├── report.tex
│   └── figures/                # EDA visualizations
│       ├── rating_distribution.png
│       ├── genre_distribution.png
│       ├── user_activity.png
│       ├── correlation_matrix.png
│       └── popularity_vs_rating.png
└── README.md
```